目录

一、 项目概述 ..... - 1 -

二、 研究背景 ..... - 1 -

三、 实验方案设计 ..... - 1 -

3.1 数据集说明 ..... - 1 -

3.2 模型说明 ..... - 2 -

3.3 基本思路 ..... - 3 -

四、 实验过程 ..... - 6 -

4.1 音频处理及特征提取 ..... - 6 -

4.2 情感识别模型训练 ..... - 10 -

4.3 模型评估 ..... - 15 -

4.4 模型导出 ..... - 19 -

4.5 GUI 展示（情感预测和文本识别） ..... - 20 -

五、 实验结果 ..... - 31 -

5.1 情感分析模型训练结果 ..... - 31 -

5.2 最终实现结果 ..... - 35 -

六、 实验总结 ..... - 37 -

6.1 小组分工 ..... - 37 -

6.2 实验心得 ..... - 37 -

七、 参考资料 ..... - 38 -

一、 项目概述

本项目实现了一个完整的中文语音情感文本识别应用。

实验使用的数据基于 ESD 数据集，选取了 Mel 倒谱系数、频谱质心、短时过零率、均方根值能量等多种语音情感相关的音频特征，经过多次训练和评估得到了一个基于 BiLSTM 的语音情感识别模型，该模型在 ESD 上的准确率达到 93%。同时，我们基于百度语音识别 API 实现了语音文本识别。

最终实现了一个功能完整、界面美观的语音情感文本识别 GUI，当用户选择对应语音音频文件后，可在界面显示音频的 wav 图和 MFCC 图，并识别得到音频的情感类别和文本内容，经过测试表明两类识别都有较好的效果。

二、 研究背景

随着信息技术的高速发展，人类与计算机之间的交互技术愈发受到研究者的关。在人机交互中，计算机往往需要具备感知情感的能力，根据操作者的不同情绪调整不同的交互方式，以最大满足操作者的需要。情感识别是利用计算机建立多种信息载体与情感度量之间的关系，以赋予计算机识别与理解人类情感的能力，包括多个方面，比如语音情感识别、面部情感识别、肢体情感识别等。

本次实验中，我们小组选择了语音情感识别这一研究方向。语音是人与人交流与传递信息的最直接和最简便的方式之一。语音信号中不仅包含言语内容信息，而且包含着丰富的副语言信息（情感、年龄和性别等）。对于言语文本内容相同的一句话，当说话人表现的情感不同时，表达的语义可能截然不同。因此，准确地判断副语言信息中包含的情感信息不仅有助于对言语内容的理解，而且能够实现情感理解与交互。语音情感识别已成为国家人工智能发展战略布局的重要组成部分。

三、 实验方案设计

3.1 数据集说明

ESD 全称 Emotional Speech Database，是一个用于语音转换研究的情感语音数据集。数据集共包含 20 个组，由 10 个以汉语为母语和 10 个以英语为母语的人说的 350 个平行话语组成，涵盖 5 个情感类别（中性、快乐、愤怒、悲伤和惊讶）。在受控声学环境中记录了超过 29 小时的语音数据。该数据集适用于多语言和跨语言情感语音转换研究。

针对本项目分析该数据集的优异性：

① 该数据集总共包含 35000 个样本，规模较大，能提供更好的训练效果和泛化能力；

② 该数据集中每个类别的样本数都是相同的，样本均衡，有助于避免模型对某个类别的偏好；

③ 数据集同时包括中文语音和英文语音，体现了样本多样性，确保模型具有广泛的泛化能力；

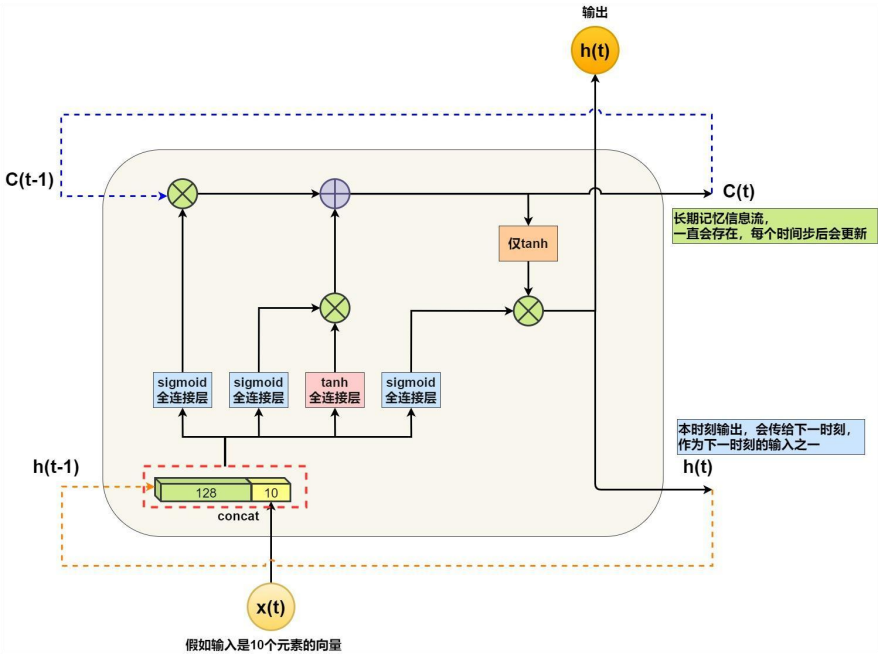
④ 每种类别语音对应的文本内容都是一致的，仅仅是说话人的情感不同，减少了文本内容带来的情感偏见，能使模型更多地关注声音情感特征。

获取链接：https://github.com/HLTSingapore/Emotional-Speech-Data

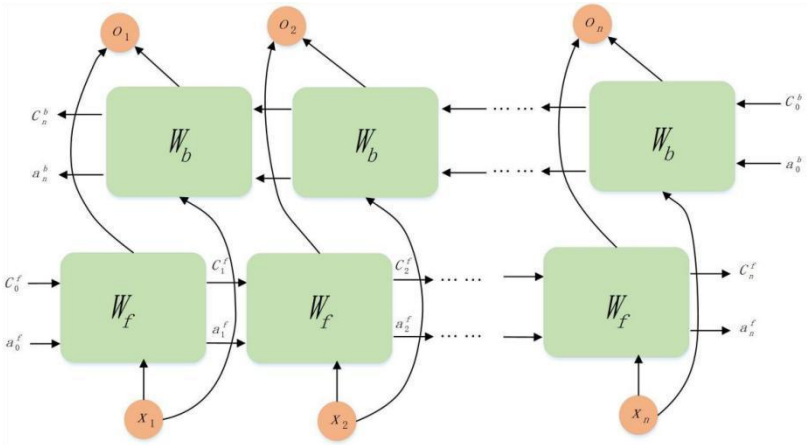
3.2 模型说明

本项目中语音情感识别使用的模型为 BiLSTM。

LSTM（Long Short-TermMemory）在循环神经网络（RNN）的基础上改进而来。与经典的 RNN 相比，LSTM 的优点在于可以有效捕捉长序列之间的语义关联，从而解决不能实现信息的长期依赖问题，缓解梯度消失或爆发的现象。LSTM 的核心结构由遗忘门、输入门、细胞状态更新、输出门四部分组成。

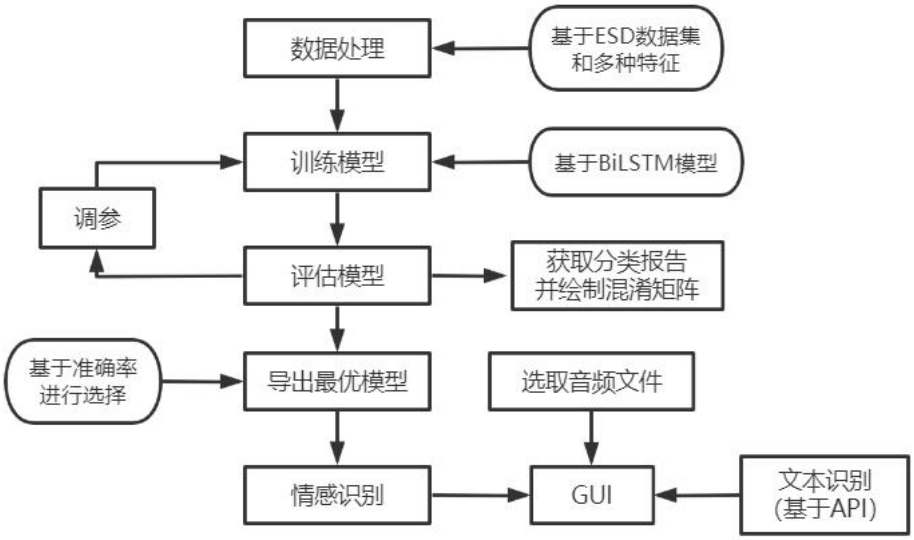


BiLSTM 神经网络结构模型是由 2 个独立的 LSTM 组成，分别以正序列和逆序列输入到 2 个 LSTM 神经网络进行特征提取，将 2 个输出向量（即提取后的特征向量）拼接后形成的情感词向量作为该词的最终特征表达。BiLSTM 的模型设计理念是让 T 时刻所得到的特征数据同时具备过去和未来之间的信息。相对于 LSTM，BiLSTM 在语音情感识别具有更好的准确性和性能。

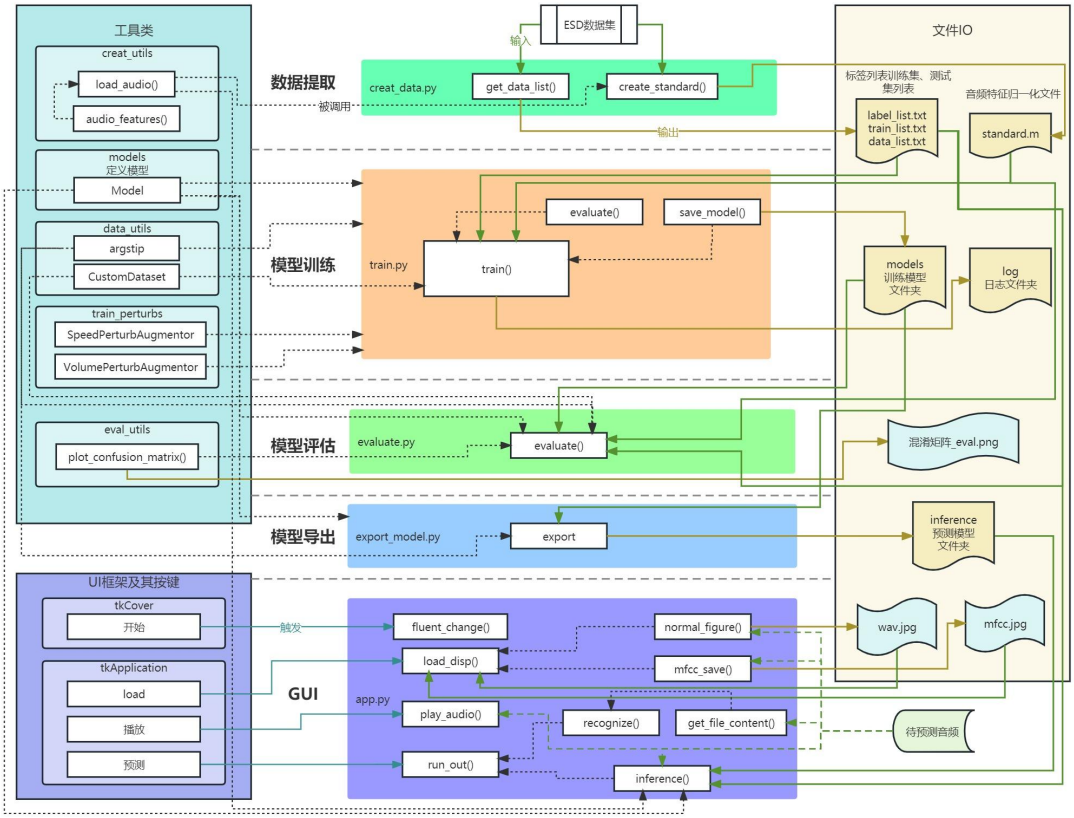


3.3 基本思路

本项目主要分为两大模块，语音情感识别模型和 GUI 实现，基本实现思路和流程图如下：



最终实现的代码逻辑如下：



（1）语音情感识别模型

对于实现一个完整的语音情感识别模型，可以分为数据预处理、特征提取、模型训练和模型评估四大部分。

① 数据预处理：

根据数据集在本地的存储路径生成音频列表文本文件，记录每条列表的路

径和情感标签，便于后续训练评估过程中读取音频。

观察 ESD 数据集中的音频样本，发现音频的长度基本在 1~3s 左右，对于训练和评估的样本，为了避免显存溢出，我们对超过 3s 的音频进行了裁剪。对于训练的样本，为了增强数据的多样性，我们进行了两种随机数据增强，包括语速增强和音量增强。

② 特征提取：

相较于传统的单一特征，我们选取了 Mel 倒谱系数、频谱质心、短时过零率、均方根值能量等多种语音情感相关的音频特征，共计 312 维，更能反应语音的情感特性。

③ 模型训练：

定义 BiLSTM 模型，对预处理后的音频数据进行逐个 epoch 迭代训练，这里使用使用随机梯度下降（SGD）作为优化方法，并设置学习率、动量和权重衰减。在每个 epoch 中，计算模型输出、损失和准确率，并进行反向传播和参数更新。每个 epoch 结束后，使用测试集对模型进行评估，计算准确率。这里保存相关数据到日志文件，可对模型训练的过程进行可视化。

④ 模型评估：

使用测试集对训练后的模型进行评估，获取分类报告并绘制混淆矩阵，比较各参数下的模型性能，选取准确率最高的模型。

④ 模型导出：

读取训练模型，将训练过的模型加上一层 softmax 函数之后，导出为预测模型，用于请按分类的预测。

（2）GUI 实现

对于 GUI 实现，可以分为 GUI 界面制作和事件响应两部分。

① GUI 界面制作：

实现美观简洁的 GUI 界面，包括设置背景图片、文字格式和用户按钮等，我们还考虑了鼠标悬停时按钮的变化。

② 事件响应：

当用户点击相应按钮时，需要触发相应函数，执行情感识别、文字识别、绘制特征图等操作，与情感识别模型、百度 API 建立良好的连接。

四、 实验过程

4.1 音频处理及特征提取

• 首先，查看数据集的基本情况：

该数据集共分为 20 个子集（0001-0020），每个子集下有五个情感文件夹和一个记录了该子集下所有音频属性（编号、文本内容、情感）的文本。每个情感文件夹下又有训练音频 300 个，测试音频 30 个，评价音频 20 个。

• 数据集预处理及音频提取 /create_data.py

该部分由 create_data.py 和 creat_utils 完成。

首先，遍历 20 个 ESD 子目录下的文件夹：

```
#子目录
aps = ['0001', '0002', '0003', '0004', '0005', '0006', '0007', '0008', '0009', '0010',
       '0011', '0012', '0013', '0014', '0015', '0016', '0017', '0018', '0019', '0020']
```

在 ESD 目录下创建统计文件用于后期处理：

```
#创建统计文件
f_train = open(os.path.join(list_path, 'train_list.txt'), 'w', encoding='utf-8')
f_test = open(os.path.join(list_path, 'test_list.txt'), 'w', encoding='utf-8')
```

```
f_label = open(os.path.join(list_path, 'label_list.txt'), 'w', encoding='utf-8')
label_first = True
```

依次遍历五种情感文件夹及音频。

```
for path in aps:
    audio_p0 = os.path.join(audio_path, path)
    audios = os.listdir(audio_p0)
    #angry-0,happy-1,neutral-2,sad-3,surprise-4
```

如遍历到 angry 时，就将其目录名称写入 label，然后访问该文件夹，遍历子文件夹 train 并将路径名和 label 索引写入 train_list，接着遍历 test 文件夹下的所有音频，将路径和 label 索引写入 test_list，遍历结束后 label_idx+1，访问下一个情感音频继续以上操作。在访问第一个 ESD 子文件夹完毕后，将 label_first 设为 false，表示 label_list 不再写入。

```
label_idx = 0
for i in range(len(audios)):
    if(audios[i][-1]=='t'):
        continue

    if(label_first):
        f_label.write(f'{audios[i]}\n')

    p_sounds = os.path.join(audio_p0, audios[i])

    train_sounds = os.listdir(os.path.join(p_sounds, "train"))
    for sound in train_sounds:
        sound_path = os.path.join(p_sounds, "train", sound).replace('\\', '/')
        f_train.write('%s\t%d\n' % (sound_path, label_idx))

    test_sounds = os.listdir(os.path.join(p_sounds, "test"))
    for sound in test_sounds:
        sound_path = os.path.join(p_sounds, "test", sound).replace('\\', '/')
        f_test.write('%s\t%d\n' % (sound_path, label_idx))

    label_idx += 1

if(label_first):
    label_first = False
    f_label.close()
```

全部遍历完毕后，在数据集目录下生成相关文件：

```
get_data_list('ESD', 'ESD')
```

label_list.txt	2023/5/22 22:43
train_list.txt	2023/5/22 22:43
test_list.txt	2023/5/22 22:43

随后，逐行遍历 train_list 下的音频，调用 creat_utils 下的 loader 文件提取音频特征，音频特征提取完毕后生成归一化文件并存储：

```
from creat_utils.loader import *
```

```
# 生成归一化文件
def create_standard(list_path, scaler_path):
    #遍历所有用于训练的音频
        with open(os.path.join(list_path, 'train_list.txt'), 'r', encoding='utf-8') as f:
        lines = f.readlines()
    #获取其音频特征
    data = []
    for line in tqdm(lines):
        path, label = line.split('\t')
        data.append(load_audio(path, mode='infer'))
    #拟合数据
    scaler = StandardScaler().fit(data)
    #存储在指定路径下
    joblib.dump(scaler, scaler_path)
```

- **音频特征提取　　/creat_utils**

Loader.py 用于加载并预处理音频，feature.py 用于提取一段音频的特征并返回。

Loader 接受音频路径及 mode 作为参数，在预处理部分，mode 为 infer，表示不对音频进行随即裁剪，直接调用 features 获取音频特征：

```
# 读取音频数据
wav, sr_wav = librosa.load(audio_path, sr=sr)
```

```
 # 获取音频特征
features = audio_features(X=wav, sample_rate=sr)
return features
```

在后续的模型训练及评估过程中，首先读取音频数据，然后会对音频进行 3s 的随即裁剪及数据增强，裁剪完毕后再获得新音频的特征：

```
# 随机裁剪
num_wav_samples = wav.shape[0]
# 计算样本数
num_chunk_samples = int(chunk_duration * sr)
```

```
if mode == 'train':
    if num_wav_samples > num_chunk_samples + 1:
        start = random.randint(0, num_wav_samples - num_chunk_samples - 1)
        stop = start + num_chunk_samples
        wav = wav[start:stop]

        # 一定概率再次裁剪
        if random.random() > 0.5:
            crop_length = random.randint(1, sr // 4) #定义一个随机数
            wav[:crop_length] = 0 #用 0 填充
            wav = wav[:-crop_length] #删除部分样本

    # 数据增强
    if augmentors is not None:
        for key, augmentor in augmentors.items():
            if key == 'specaug':
                continue
            wav = augmentor(wav)

elif mode == 'eval':
    # 为避免显存溢出，只裁剪指定长度
    if num_wav_samples > num_chunk_samples + 1:
        wav = wav[:num_chunk_samples]
```

Features.py 提取特征的主要内容如下：考虑到实际应用及需要，我们获取了音频的很多特征（如语音频率、音高、频谱质心、谱平面、色谱图、MFCC、梅尔频率等）

在获取所有音频特征后，将大部分特征值存储在 features 数组中：

```
features = np.array([
    flatness, zerocr, Magnitude_mean, Magnitude_max, cent_mean, cent_std,
    cent_max, Magnitude_std, pitch_mean, pitch_max, pitch_std,
    pitch_tuning_offset, rms_mean, rms_max, rms_std
])
```

最后，将特征值与 MFCC、色谱图、梅尔频率、谱对比度进行连接，并将结果转换为 np.float32 类型。

```
    features = np.concatenate((features, mfccs, mfccs_std, mfcc_max, chroma, mel, contrast)).astype(np.float32)
    return features
```

所有音频特征获取完毕并记录后，在 create_data 中调用 standardscaler 创建一个标准化器，拟合数据，将其标准化保存到 ESD 目录下用于后续处理：

```
    #拟合数据
    scaler = StandardScaler().fit(data)
    #存储在指定路径下
    joblib.dump(scaler, scaler_path)
    create_standard('ESD', 'ESD/standard.m')
```

standard.m	2023/5/22 22:43	M 文件	8 KB

4.2 情感识别模型训练

在上一步的数据集预处理和特征提取后，开始训练模型。

/train.py

主要分为三个部分：

（1）解析命令行参数

使用 argparse 库解析命令行传入的参数，并定义了一系列参数的默认取值，如：batch_size、num_workers、num_epoch、num_class 等参数。最后将设置好的参数赋值给 args 对象。

```
    parser = argparse.ArgumentParser(description=__doc__)
    add_arg = functools.partial(add_arguments, argparser=parser)
    add_arg('batch_size', int, 512, '训练的批量大小')
    add_arg('num_workers', int, 8, '读取数据的线程数量')
    add_arg('num_epoch', int, 100, '训练的轮数')
    add_arg('num_class', int, 5, '分类的类别数量')
    add_arg('learning_rate', float, 0.5, '初始学习率的大小')
    add_arg('train_list_path', str, 'ESD/train_list.txt', '训练数据的数据列表路径')
    add_arg('test_list_path', str, 'ESD/test_list.txt', '测试数据的数据列表路径')
    add_arg('scaler_path', str, 'ESD/standard.m', '测试数据的数据列表路径')
    add_arg('save_model_dir', str, 'output/models/', '模型保存的路径')
    add_arg('augment_conf_path', str, 'configs/augment.yml', '数据增强的配置文件，为 json 格式')
    add_arg('resume', str, None, '恢复训练的模型文件夹，当为 None 则不使用恢复模型')
    add_arg('pretrained_model', str, None, '预训练模型的模型文件夹，当为 None 则不使用预训练模型')
    add_arg('device_type', str, 'cuda', '设备类型 cuda/cpu')
    args = parser.parse_args()
```

在随后的训练过程中，我们主要对 batch_size、num_epoch、learning_rate 进行了调整来获得最优模型。图示数据为测试过程中得到的较优模型参数。

（2）定义训练函数

首先，通过数据增强器对训练音频进行随机语速增强和音量改变。

该部分通过调用 train_perturbs 下的 speed_perturb（语速增强）和 volume_perturb（音量调节）实现。

```
from train_perturbs.speed_perturb import SpeedPerturbAugmentor
from train_perturbs.volume_perturb import VolumePerturbAugmentor
```

```
 # 获取数据增强器
augmentors = None
if args.augment_conf_path is not None:
    augmentors = {}
    with open(args.augment_conf_path, encoding="utf-8") as fp:
        configs = yaml.load(fp, Loader=yaml.FullLoader)
    augmentors['speed'] = SpeedPerturbAugmentor(**configs['speed'])
    augmentors['volume'] = VolumePerturbAugmentor(**configs['volume'])
```

随后，获取 train_dataset 和 eval_dataset 用于训练和评价：

```
# 获取数据
train_dataset = CustomDataset(
    args.train_list_path,
    scaler_path=args.scaler_path,
    mode='train',
    sr=16000,
    chunk_duration=3,
    augmentors=augmentors
)
train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=args.batch_size,
    num_workers=args.num_workers,
    shuffle=True
)
# 测试数据
eval_dataset = CustomDataset(
    args.test_list_path,
    scaler_path=args.scaler_path,
```

```
        mode='eval',
        sr=16000,
        chunk_duration=3
    )
    eval_loader = DataLoader(
        dataset=eval_dataset,
        batch_size=args.batch_size,
        num_workers=args.num_workers
    )
```

结合优化方法生成一个调度器 scheduler，让学习率进行动态衰减。

```
# 获取优化方法
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=args.learning_rate,
    momentum=0.9,
    weight_decay=5e-4
)
# 获取学习率衰减函数
scheduler = CosineAnnealingLR(optimizer, T_max=args.num_epoch)
```

随后，进入一个循环内部，每轮循环中内部嵌套一个遍历训练数据的循环，利用 PyTorch 内置的 DataLoader 进行数据载入。

```
for epoch in range(last_epoch, args.num_epoch):
    loss_sum = []
    accuracies = []
    start = time.time()
    for batch_id, (audio, label) in enumerate(train_loader):
```

读取的训练数据进行前向传播，计算损失值与梯度值，最后利用优化器进行参数的更新，同时计算每个 batch 的训练损失和准确率，并打印相关信息。

```
for batch_id, (audio, label) in enumerate(train_loader):
    audio = audio.to(device)
    label = label.to(device).long()
    output = model(audio)
    # 计算损失值
    loss = criterion(output, label)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    # 计算准确率
    output = output.data.cpu().numpy()
```

```
        output = np.argmax(output, axis=1)
        label = label.data.cpu().numpy()
        acc = np.mean((output == label).astype(int))
        accuracies.append(acc.item())
        loss_sum.append(loss.item())
        # 多卡训练只使用一个进程打印
        if batch_id % 10 == 0:
                eta_sec = ((time.time() - start) * 1000) * (sum_batch - (epoch - last_epoch) * len(train_loader) - batch_id)
            eta_str = str(timedelta(seconds=int(eta_sec / 1000)))
            print(f'[{datetime.now()}] '
                  f'Train epoch [{epoch}/{args.num_epoch}], '
                  f'batch: [{batch_id}/{len(train_loader)}], '
                  f'loss: {(sum(loss_sum) / len(loss_sum)):.5f}, '
                  f'accuracy: {(sum(accuracies) / len(accuracies)):.5f}, '
                  f'lr: {scheduler.get_lr()[0]:.8f}, '
                  f'eta: {eta_str}')
            writer.add_scalar('Train/Loss', loss.item(), train_step)
            writer.add_scalar('Train/Accuracy', (sum(accuracies) / len(accuracies)), train_step)
            train_step += 1
        start = time.time()
```

之后，train()函数会调用 evaluate()函数，将当前模型在测试集上的准确率计算出来。

```
# 执行评估和保存模型
s = time.time()
acc = evaluate(model, eval_loader)
eta_str = str(timedelta(seconds=int(time.time() - s)))
print('=' * 70)
print(f'[{datetime.now()}] Test {epoch}, Accuracy: {acc:.5f}, time: {eta_str}')
print('=' * 70)
```

最后，train()会调用 save_model()函数来进行训练模型和优化器参数的保存。

```
writer.add_scalar('Test/Accuracy', acc, test_step)
# 记录学习率
writer.add_scalar('Train/Learning rate', scheduler.get_lr()[0], epoch)
test_step += 1
scheduler.step()
# 保存模型
```

```
save_model(args.save_model_dir, model, optimizer, epoch)
```

（3）定义评估函数和保存模型参数

Evaluate：用于计算模型在测试集上的准确率。首先激活评估模式，将模型参数传入设备（如 cuda），利用视觉化库（visualdl）得出输出，再利用 numpy 得出各个 batch 的输出准确率，并将准确率求平均值返回。

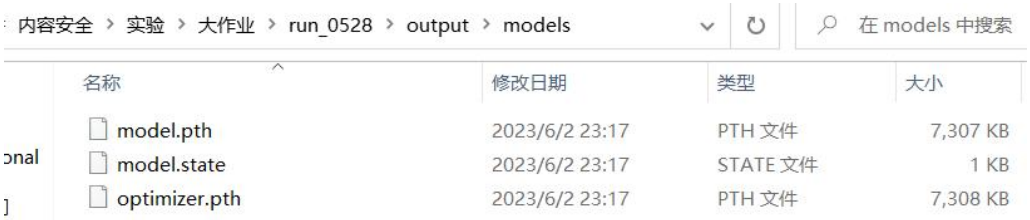
```
def evaluate(model, test_loader):
    model.eval()
    accuracies = []
    device = torch.device(args.device_type)
    for batch_id, (audio, label) in enumerate(test_loader):
        audio = audio.to(device)
        output = model(audio)
        # 计算准确率
        output = output.data.cpu().numpy()
        output = np.argmax(output, axis=1)
        label = label.data.cpu().numpy()
        acc = np.mean((output == label).astype(int))
        accuracies.append(acc.item())
    model.train()
    acc = float(sum(accuracies) / len(accuracies))
    return acc
```

save_model 函数用于保存最新的训练模型，以及训练过程中的优化器状态和当前 epoch 值。利用 PyTorch 自带的 save 函数将当前模型和优化器的参数保存下来。

```
# 保存模型
def save_model(save_path, model, optimizer, epoch):
    os.makedirs(save_path, exist_ok=True)
    torch.save(model.state_dict(), os.path.join(save_path, 'model.pth'))
    torch.save({'last_epoch': torch.tensor(epoch)}, os.path.join(save_path, 'model.state'))
    torch.save(optimizer.state_dict(), os.path.join(save_path, 'optimizer.pth'))
```

训练完毕后，结果保存在 output 目录下，用于后续的 evaluate 和 test：

```
add_arg('save_model_dir', str, 'output/models/', '模型保存的路径')
```



4.3 模型评估

• 首先，介绍 data_utils 目录下的 myDataset，该部分实现数据加载功能，用于在训练和评估时加载和预处理音频数据。

```
def __init__(self, data_list_path, scaler_path, mode='train', sr=16000, chunk_duration=3, augmentors=None):
    super(CustomDataset, self).__init__()
    # 当预测时不需要获取数据
    if data_list_path is not None:
        with open(data_list_path, 'r') as f:
            self.lines = f.readlines()
    self.mode = mode
    self.sr = sr
    self.chunk_duration = chunk_duration
    self.augmentors = augmentors
    self.scaler = joblib.load(scaler_path)
```

相关参数：音频列表文件路径、归一化器路径、当前模式（mode）、采样率、样本时长（默认 3s）和数据增强器

lines 中记录音频列表，逐一获取音频文件的路径和标签，采用 load_audio 函数提取特征，并进行归一化，最终返回每个样本对应的特征与标签。

当处理发生异常时，输出对应异常音频和异常信息。

```
def __getitem__(self, idx):
```

```
        try:
            audio_path, label = self.lines[idx].replace('\n', '').split('\t')
            # 加载并预处理音频
            features = load_audio(audio_path, mode=self.mode, sr=self.sr,
                                  chunk_duration=self.chunk_duration, augmentors=self.augmentors)
            # 转化为二维数组后进行归一化
            features = self.scaler.transform(features.reshape(1, -1))
            features = np.squeeze(features).astype(np.float32)
            label = np.array(int(label), dtype=np.int64)
            return features, label
        except Exception as ex:
                print(f"[{datetime.now()}] 数据：{self.lines[idx]} 出错，错误信息: {ex}", file=sys.stderr)
            rnd_idx = np.random.randint(self.__len__())
            return self.__getitem__(rnd_idx)
```

• 评估模型效果 /evaluate：

该部分分为两步：

（1）解析命令行参数：使用 argparse 库解析命令行传入的参数，并定义了一系列参数的默认取值

```
parser = argparse.ArgumentParser(description=__doc__)
add_arg = functools.partial(add_arguments, argparser=parser)
add_arg('batch_size',       int,    32,                        '训练的批量大小')
add_arg('num_workers',      int,    4,                         '读取数据的线程数量')
add_arg('num_class',        int,    5,                         '分类的类别数量')
add_arg('test_list_path',   str,    'ESD/test_list.txt',       '测试数据的数据列表路径')
add_arg('label_list_path',  str,    'ESD/label_list.txt',      '标签列表路径')
add_arg('scaler_path',      str,    'ESD/standard.m',          '测试数据的数据列表路径')
add_arg('model_path',       str,    'output/models/model.pth', '模型保存的路径')
add_arg('device_type',      str,    'cpu',                     '设备类型 cuda/cpu')
args = parser.parse_args()
```

用于测试的数据由预处理部分，用于测试的模型由训练部分生成。

（2）加载用于测试的音频文件进行评估

首先通过加载 test_list 中的音频文件并预处理获取评估数据：

```
# 获取评估数据
eval_dataset = CustomDataset(args.test_list_path,
```

```
                                scaler_path=args.scaler_path,
                                mode='eval',
                                sr=16000,
                                chunk_duration=3)
    eval_loader = DataLoader(dataset=eval_dataset,
                             batch_size=args.batch_size,
                             num_workers=args.num_workers)
```

然后获取分类标签：

```
    with open(args.label_list_path, 'r', encoding='utf-8') as file:
        lines = file.readlines()
        class_labels = []
        for line in lines:
            class_labels.append(line.replace('\n', ''))
```

随后导入模型，加载参数。将音频特征作为输入模型进行预测，输出每个样本对应每种标签的概率分布，取概率最大的标签为最终预测标签。迭代结束后得到完整的预测结果 pred_labels 和真实结果 ture_labels。

```
    # 获取模型
    device = torch.device(args.device_type)
    model = Model(num_class=args.num_class)
    # 读取训练好的模型参数并加载到模型中
    weights = torch.load(args.model_path, map_location=torch.device(args.device_type))
    model.load_state_dict(weights)
    model.to(device)

    # 开始进行评估
    model.eval()
    print('开始评估...')
    pred_labels, ture_labels = [], []
    for batch, (audio_feature, ture_label) in enumerate(eval_loader):
        audio_feature = audio_feature.to(device)
        # 进行预测，得到每种标签的概率分布
        output = model(audio_feature)
        # 取概率最大的标签为最终预测标签
        pred_label = np.argmax(output.data.cpu().numpy(), axis=1)
        pred_labels.extend(pred_label)
        # 真实标签
        ture_labels.extend(ture_label.data.cpu().numpy().tolist())
```

最后，直接采用 classification_report 获取模型评估报告，得到总体准确率、

精确率、召回率和 f1 值，且可以看到每种类别的样本分布均匀。

这里给出最终模型的评估结果：

	precision	recall	f1-score	support
Angry	0.9393	0.9033	0.9210	600
Happy	0.8525	0.9250	0.8873	600
Neutral	0.9557	0.9700	0.9628	600
Sad	0.9879	0.9533	0.9703	600
Surprise	0.9195	0.8950	0.9071	600
accuracy			0.9293	3000
macro avg	0.9310	0.9293	0.9297	3000
weighted avg	0.9310	0.9293	0.9297	3000

• 混淆矩阵的绘制由 eval_utils 目录下的 matrix.py 实现：（以最优模型为例）

```
def plot_confusion_matrix(matrix, class_labels, type='precision', save_path = 'output/confusion_matrix.png', show=False):
```

matrix 为输入的混淆矩阵， class_labels 是类别名，type 可为'recall'和'precision'两种值，save_path 为图片保存路径，show 为是否直接展示图片。

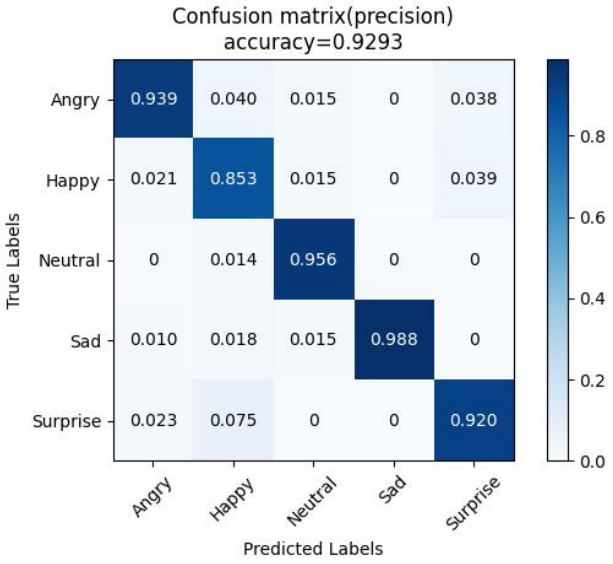
通过将原混淆矩阵中[y][x]的元素除以 x 列或 y 行的总和计算得到精确率或召回率，使用 1e-6 作为平滑项，避免除零错误，结果记录在 prob 中：

```
 # prob 与混淆矩阵同样结构,记录目标被分为各类别的概率
rows, cols = np.shape(matrix)
prob = np.zeros((rows, cols))
for x in range(num_class):
    for y in range(num_class):
        # 计算精确度或召回率
        if type == 'recall':
            prob_xy = matrix[y][x] / (np.sum(matrix[y, :]) + 1e-6)
        else:
            prob_xy = matrix[y][x] / (np.sum(matrix[:, x]) + 1e-6)
        prob[y, x] = prob_xy
```

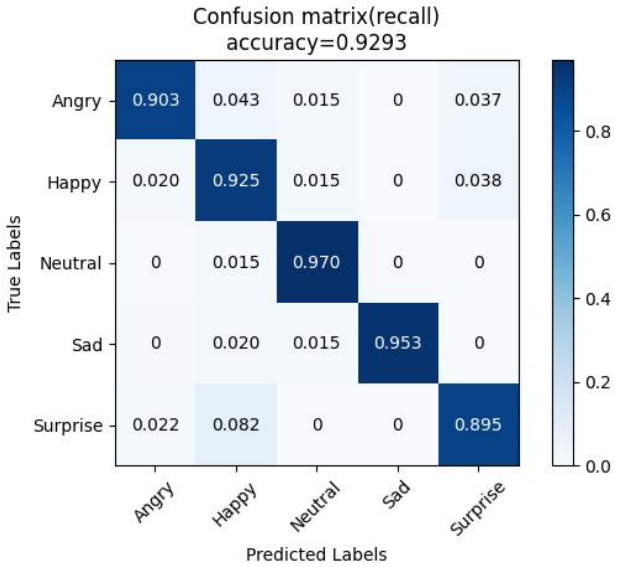
最终绘制出的混淆矩阵如下：

x 轴对应预测标签，y 轴对应真实标签（忽略概率过小的值，未在图上标出）

精确度：每[y][x]格中的值表示被预测为 x 标签的样本中真实标签为 y 的概率，可以直观地展示某类预测结果中存在误分结果的可能性，如被预测为 Surprise 的样本实际为 Angry 或 Happy 的可能性相对较大。



召回率：每[y][x]格中的值表示真实标签为 y 被预测为 x 标签的概率，可以直观地展示被误分为其他类别的可能性，如 Surprise 被误分为 Happy 的可能性相对较大。



两种混淆矩阵实际上都是更直观地展现了模型误分类的情况，实际评估中根据需求选用一种即可。

4.4 模型导出

该部分分为两步　　/export_model.py：

（1）解析命令行参数

```
parser = argparse.ArgumentParser(description=__doc__)
add_arg = functools.partial(add_arguments, argparser=parser)
add_arg('model_path',       str,    'output/models/model.pth',      '模型保存的路径')
add_arg('save_path',      str,   'output/inference/inference.pth',  '模型保存的路径')
args = parser.parse_args()
```

（2）保存模型

通过 model_path 的模型路径读取训练出来的模型，在模型中加上 softmax 进行归一化，并保存为预测模型，供后续预测时使用。

```
# 获取模型
model = Model(num_class = 5)
model.load_state_dict(torch.load(args.model_path))
# 加上 Softmax 函数
model = torch.nn.Sequential(model, torch.nn.Softmax())

# 保存预测模型
os.makedirs(os.path.dirname(args.save_path), exist_ok=True)
torch.save(model, args.save_path)
```

4.5 GUI 展示（情感预测和文本识别）

1. 实现 GUI 界面

（1）创建应用程序主窗口（tk_main 函数）

由 main 函数直接调用。

创建名为 root 的 Tk()类，即主窗口，根据屏幕的宽高和设定的窗口的宽高，将窗口位置显示在屏幕正中间，创建两个自定义框架类，cover 对应应用初始首页，app 对应功能界面，并将 root 作为它们的 master 传入，以实现框架在窗口上的显示，最后启动窗口循环，使窗口保持可见并响应用户交互。

```
def tk_main():
    global root,cover,app
    root = Tk()
    sw = root.winfo_screenwidth()
    sh = root.winfo_screenheight()
    ww = bg_weight
    wh = bg_height
    x = (sw - ww) / 2
    y = (sh - wh) / 2
```

```python

    root.title("语音情感文本识别应用")
    root.geometry("%dx%d+%d+%d" % (ww, wh, x, y))
    root.resizable(False, False)

    # 初始化音频播放器
    mixer.init()

    cover = tkCover(master=root)
    app = tkApplication(master=root)
    # 窗口循环
    root.mainloop()
```

（2）应用初始首页（tkCover 类）

① __init__()初始化函数：

继承父类 Frame 的初始化，使用 place()方法将自身 Frame 框架以设定好的尺寸(1280,800)放到界面上，并调用 createWidget()函数创建窗口中的部件。

```python
bg_weight = 1280
bg_height = 800
root = None
cover = None
app = None

class tkCover(Frame):
    ww = bg_weight
    wh = bg_height

    def __init__(self, master=None):
        super().__init__(master)
        self.master = master
        self.place(width=bg_weight,height=bg_height)
        self.createWidget()
```

② get_img ()函数：

用来读取图片，调整图片大小并转化为 tkinter 能够直接使用的格式，在后续构造内部组件时使用。

```python
def get_img(self, filename, width, height):
    im = Image.open(filename).resize((width, height))
    im = ImageTk.PhotoImage(im)
    return im
```

③ createWidget()函数：用于 Frame 框架内部组件的构造。

设置背景图片：

创建一个自身的内部类 Canvas，在 Frame 框架内创建一个大小一致的画布，调用 get_img 方法加载背景图片，调整宽度和高度与 Frame 框架一致，并放置在画布的中心位置。

```
def createWidget(self):
    # 设置背景图片
    self.canvas_root = Canvas(self, width=self.ww, height=self.wh)
    self.im_root0 = self.get_img(back0_path, self.ww, self.wh)
    self.canvas_root.create_image(self.ww/2, self.wh/2, image=self.im_root0)
    self.canvas_root.pack()
```

绘制应用标题：

虽然 tkinter 自带 Label 类可以用来显示文本，但是这样显示出来的文本一定有个背景颜色，并且背景颜色无法设为透明，很不美观。所以这里选择直接利用 create_text()方法在画布 canvas_root 上画文字。

```
# 画布文字
self.label1t0 = self.canvas_root.create_text(640+2,260+2,text='语音情感文本识别应用',font=('楷体',40,'bold'),fill='gray')
self.label1t = self.canvas_root.create_text(640,260,text='语音情感文本识别应用',font=('楷体',40,'bold'),fill='black')
```

在画布上画两次相同的文字，两次文字的坐标略微错开，且第一次文字设为灰色，即可实现下图这样类似阴影的标题效果：

设置“开始”按钮：

tkinter 自带的 Button 类可以实现按钮的功能，但是同样非常不美观，背景颜色同样无法设为透明，而且无法对鼠标悬停作出反应。

这里使用了一个比较巧妙的方法自己实现一个按钮，首先根据设定的按钮边界，在画布上画两个矩形作为按钮外部框架，并在框架中间写上文本，实现按钮的外观。

```
# 用画布仿制透明按钮
self.bt1_left = 400 ; self.bt1_top = 550; self.bt1_right = 880; self.bt1_bottom = 600
self.bt1r1 = self.canvas_root.create_rectangle(self.bt1_left,self.bt1_top,self.bt1_right,self.bt1_bottom,width=2,outline='black')# 按钮外框
```

```
    self.bt1r2 = self.canvas_root.create_rectangle(self.bt1_left+3,self.bt1_top+3,self.bt1_right-3,self.bt1_bottom-3,width=2,outline='black')# 按钮内框
    self.bt1t = self.canvas_root.create_text((self.bt1_left+self.bt1_right)/2,(self.bt1_top+self.bt1_bottom)/2,text='开 始',font=('楷体',20,'bold'),fill='black')# 按钮显示文本
```

按钮初始状态如下：

然后将矩形框架范围内的点击事件和鼠标经过与具体函数相绑定，以实现透明按钮和鼠标经过按钮时的反馈。

绑定鼠标左键点击事件到画布，当点击事件发生时，将调用 bind_click()函数。在 bind_click()中检查鼠标点击的位置是否在“开始”按钮的范围内，若在按钮的范围内则调用 fluent_change()函数，切换到功能界面。

绑定鼠标经过事件到画布，当鼠标经过事件发生时，将调用 motion_all()函数。在 motion_all()中，对鼠标的位置进行判断，当鼠标悬停在我们自己做的按钮上时，改变按钮颜色为白色；若鼠标移动到按钮外就恢复按钮颜色为黑色。

```
    self.canvas_root.bind('<Button-1>',lambda event:bind_click(event))# 关联鼠标点击事件
    self.canvas_root.bind('<Motion>',lambda event:motion_all(event))# 关联鼠标经过事件

    def bind_click(event):# 点击响应函数
        if self.bt1_left<=event.x<=self.bt1_right and self.bt1_top<=event.y<=self.bt1_bottom:# 响应的位置
            fluent_change()

    def motion_all(event):# 鼠标经过响应函数
        if self.bt1_left<=event.x<=self.bt1_right and self.bt1_top<=event.y<=self.bt1_bottom:# 响应的位置
            self.canvas_root.itemconfigure(self.bt1r1,outline='white')# 重设外框颜色
            self.canvas_root.itemconfigure(self.bt1r2,outline='white')# 重设内框颜色
            self.canvas_root.itemconfigure(self.bt1t,fill='white')# 重设显示文本颜色
        else:
            self.canvas_root.itemconfigure(self.bt1r1,outline='black')# 恢复外框默认颜色
            self.canvas_root.itemconfigure(self.bt1r2,outline='black')# 恢复内框默认颜色
            self.canvas_root.itemconfigure(self.bt1t,fill='black')# 恢复显示文本默认颜色
```

当鼠标经过或悬停在按钮上时的按钮状态如下：

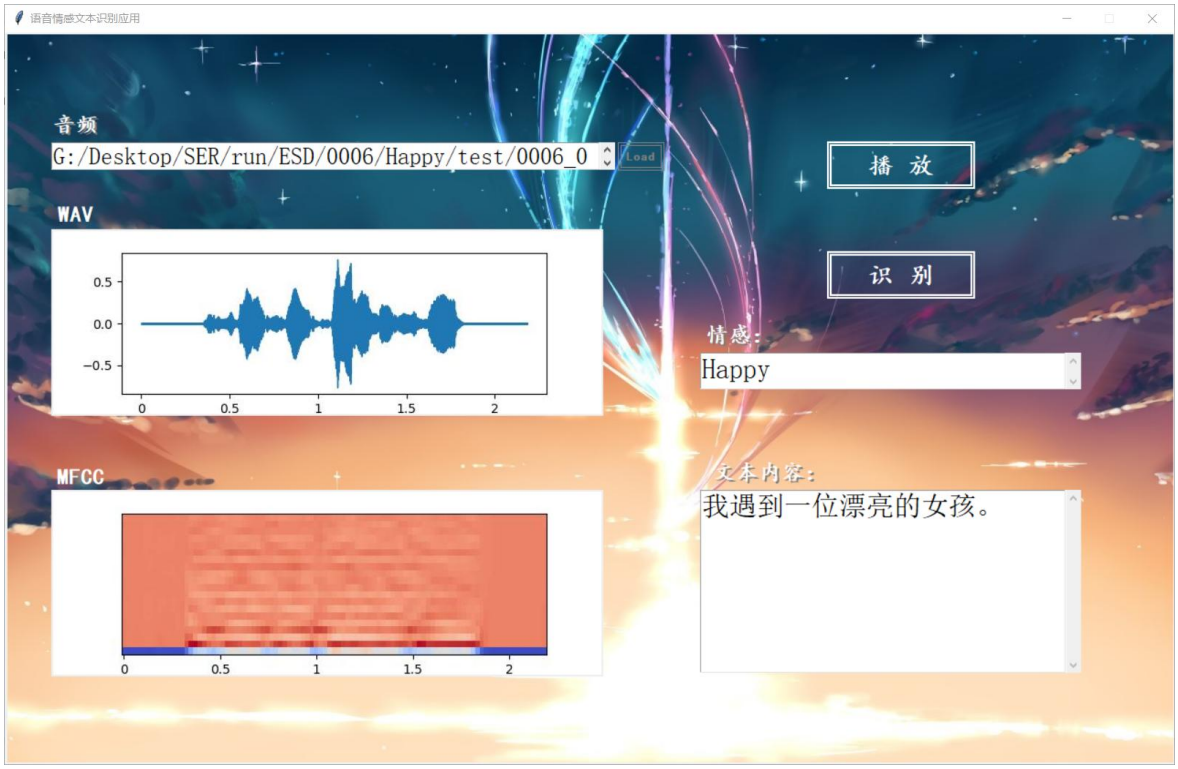
（3）界面切换（fluent_change 函数）

该函数由鼠标左键点击初始首页的“开始”按钮触发，实现将首页切换为功能界面的功能。

在 fluent_list 列表中定义一系列的平滑过渡值，每 20ms 按照 fluent_list 中对应的一定比例向左移动一次 cover 和 app 两个框架，逐步实现两个界面的切换。fluent_list 列表中最后的负值实现了些许的回弹，使界面之间的切换更丝滑。

```
def fluent_change(i=0):
    global cover, app, root
    fluent_list = [0.01,0.01,0.01,0.01,0.03,0.03,0.05,0.05,0.09,0.09,0.15,0.15,0.09,0.09,0.05,0.04,0.03,0.02,0.01,0.01,0.01,-0.01,-0.01,-0.01]
    imx = len(fluent_list) - 1
    cover.place(x=int(cover.place_info()['x'])-bg_weight*fluent_list[i])
    app.place(x=int(app.place_info()['x'])-bg_weight*fluent_list[i])
    if i<imx:
        root.after(20,fluent_change,i+1)  # 20ms 移动一次
```

（4）应用功能界面（tkApplication 类）



① __init__()初始化函数和 get_img ()函数：

实现的功能与 tkCover 类中一致，实现了初始化和读取图片的功能。

```python
class tkApplication(Frame):
    ww = bg_weight
    wh = bg_height
    radio_path = None
    play_status = False

    def __init__(self, master=None):
        super().__init__(master)
        self.master = master
        self.place(width=bg_weight,height=bg_height,x=bg_weight)
        self.createWidget()

    def get_img(self, filename, width, height):
        im = Image.open(filename).resize((width, height))
        im = ImageTk.PhotoImage(im)
        return im
```

② createWidget()函数：用于 Frame 框架内部组件的构造。

调用 get_img 方法设置背景图片：

```python
def createWidget(self):
    ### app 中所有组件必须定义为 app 的属性

    # 设置背景图片
    self.canvas_root = Canvas(self, width=self.ww, height=self.wh)
    self.im_root0 = self.get_img(back_path, self.ww, self.wh)
    self.canvas_root.create_image(self.ww/2, self.wh/2, image=self.im_root0)
    self.canvas_root.pack()
```

设置按钮：

采用设置“开始”按钮的方法，绘制“Load”“识别”“播放”3 个按钮。

```python
# 用画布仿制透明按钮
self.bt1_left = 670 ; self.bt1_top = 120; self.bt1_right = 720; self.bt1_bottom = 150
self.bt1r1 = self.canvas_root.create_rectangle(self.bt1_left,self.bt1_top,self.bt1_right,self.bt1_bottom,width=1,outline='gray')# 按钮外框
self.bt1r2 = self.canvas_root.create_rectangle(self.bt1_left+3,self.bt1_top+3,self.bt1_right-3,self.bt1_bottom-3,width=1,outline='gray')# 按钮内框
self.bt1t = self.canvas_root.create_text((self.bt1_left+self.bt1_right)/2,(self.bt1_top+self.bt1_bottom)/2,text='Load',font=('楷体',10,'bold'),fill='gray')# 按钮显示文本
self.bt2_left = 900 ; self.bt2_top = 240; self.bt2_right = 1060; self.bt2_bottom = 290
self.bt2r1 = self.canvas_root.create_rectangle(self.bt2_left,self.bt2_top,self.bt2_right,self.bt2_bottom,width=2,outline='white')# 按钮外框
```

```
    self.bt2r2 = self.canvas_root.create_rectangle(self.bt2_left+3,self.bt2_top+3,self.bt2_right-3,self.bt2_bottom-3,width=2,outline='white')# 按钮内框
    self.bt2t = self.canvas_root.create_text((self.bt2_left+self.bt2_right)/2,(self.bt2_top+self.bt2_bottom)/2,text='识 别',font=('楷体',20,'bold'),fill='white')# 按钮显示文本
    self.bt3_left = 900 ; self.bt3_top = 120; self.bt3_right = 1060; self.bt3_bottom = 170
    self.bt3r1 = self.canvas_root.create_rectangle(self.bt3_left,self.bt3_top,self.bt3_right,self.bt3_bottom,width=2,outline='white')# 按钮外框
    self.bt3r2 = self.canvas_root.create_rectangle(self.bt3_left+3,self.bt3_top+3,self.bt3_right-3,self.bt3_bottom-3,width=2,outline='white')# 按钮内框
    self.bt3t = self.canvas_root.create_text((self.bt3_left+self.bt3_right)/2,(self.bt3_top+self.bt3_bottom)/2,text='播 放',font=('楷体',20,'bold'),fill='white')# 按钮显示文本
```

绑定鼠标左键点击事件和经过事件到画布，当点击事件发生时，将调用 bind_click()函数；当鼠标经过事件发生时，将调用 motion_all()函数。

bind_click()函数中判断鼠标点击的位置，点击“Load”按钮时，调用 load_disp()函数，读取指定路径的音频，并且显示该音频的 wav 图和 mfcc 图；点击“播放”按钮时，调用 play_audio()函数，播放音频；点击“识别”按钮时，调用 run_out()函数，执行音频的情感预测和文本识别，并将识别结果显示在相应的自定义文本框中。

```
    self.canvas_root.bind('<Button-1>',lambda event:bind_click(event))# 关联鼠标点击事件
    self.canvas_root.bind('<Motion>',lambda event:motion_all(event))# 关联鼠标经过事件

    def bind_click(event):# 点击响应函数
        if self.bt1_left<=event.x<=self.bt1_right and self.bt1_top<=event.y<=self.bt1_bottom:# 响应的位置
            self.load_disp()
        if self.bt2_left<=event.x<=self.bt2_right and self.bt2_top<=event.y<=self.bt2_bottom:# 响应的位置
            self.run_out()
        if self.bt3_left<=event.x<=self.bt3_right and self.bt3_top<=event.y<=self.bt3_bottom:# 响应的位置
            self.play_audio()
```

motion_all()函数中对鼠标的位置进行判断，当鼠标悬停在相应按钮上时，改变按钮颜色；若鼠标移动到按钮外就恢复按钮初始颜色。

```
    def motion_all(event):# 鼠标经过响应函数
        if self.bt1_left<=event.x<=self.bt1_right and self.bt1_top<=event.y<=self.bt1_bottom:# 响应的位置
            self.canvas_root.itemconfigure(self.bt1r1,outline='white')# 重设外框颜色
```

```
            self.canvas_root.itemconfigure(self.bt1r2,outline='white')# 重设内框颜色
            self.canvas_root.itemconfigure(self.bt1t,fill='white')# 重设显示文本颜色
        else:
            self.canvas_root.itemconfigure(self.bt1r1,outline='gray')# 恢复外框默认颜色
            self.canvas_root.itemconfigure(self.bt1r2,outline='gray')# 恢复内框默认颜色
            self.canvas_root.itemconfigure(self.bt1t,fill='gray')# 恢复显示文本默认颜色
        if self.bt2_left<=event.x<=self.bt2_right and self.bt2_top<=event.y<=self.bt2_bottom:# 响应的位置
            self.canvas_root.itemconfigure(self.bt2r1,outline='yellow')# 重设外框颜色
            self.canvas_root.itemconfigure(self.bt2r2,outline='yellow')# 重设内框颜色
            self.canvas_root.itemconfigure(self.bt2t,fill='yellow')# 重设显示文本颜色
        else:
            self.canvas_root.itemconfigure(self.bt2r1,outline='white')# 恢复外框默认颜色
            self.canvas_root.itemconfigure(self.bt2r2,outline='white')# 恢复内框默认颜色
            self.canvas_root.itemconfigure(self.bt2t,fill='white')# 恢复显示文本默认颜色
        if self.bt3_left<=event.x<=self.bt3_right and self.bt3_top<=event.y<=self.bt3_bottom:# 响应的位置
            self.canvas_root.itemconfigure(self.bt3r1,outline='yellow')# 重设外框颜色
            self.canvas_root.itemconfigure(self.bt3r2,outline='yellow')# 重设内框颜色
            self.canvas_root.itemconfigure(self.bt3t,fill='yellow')# 重设显示文本颜色
        else:
            self.canvas_root.itemconfigure(self.bt3r1,outline='white')# 恢复外框默认颜色
            self.canvas_root.itemconfigure(self.bt3r2,outline='white')# 恢复内框默认颜色
            self.canvas_root.itemconfigure(self.bt3t,fill='white')# 恢复显示文本默认颜色
```

设置文本框和滚动条：

使用 Text()创建文本框控件，Scrollbar()创建滚动条控件，通过设置文本框的 yscrollcommand 属性和滚动条的 command 属性将两者绑定，并将两者放置在指定位置。在功能界面共设置了 3 个这样的文本框，分别用于显示音频路径、情感预测结果和文本识别结果。

```
# 可滚动多行文本 text
t0x = 50; t0y = 120; t0w = 600; t0h = 30
self.text0 = Text(self, font="song -26", bg="white")
self.text0.place(x=t0x, y=t0y, width=t0w, height=t0h)
self.sl0 = Scrollbar(self)
self.text0['yscrollcommand'] = self.sl0.set
self.sl0['command'] = self.text0.yview
self.sl0.place(x=t0x+t0w, y=t0y, height=t0h)
```

③ load_disp()函数：用于读取指定路径的音频，并且显示该音频的 wav 图和 mfcc 图。

调用 askopenfilename()函数弹出文件选择对话框，让用户选择一个音频文件，并将选择的文件路径显示在文本框中；根据所选音频路径调用 normal_figure()和 mfcc_save()函数，绘制并保存音频的 wav 图和 MFCC 图。

最后读取生成的图像文件，展示在界面左侧的对应位置。

```
def load_disp(self):
    self.text0.delete(1.0, END)
    self.radio_path = askopenfilename()
    self.text0.insert(1.0, self.radio_path)
    mfcc_save(self.radio_path)
    normal_figure(self.radio_path)

    self.im00 = Image.open("sources/mfcc.jpg")
    self.im01 = ImageTk.PhotoImage(self.im00)
    self.img0 = Label(self, image=self.im01)
    self.img0.place(x=50, y=500)

    self.im10 = Image.open("sources/wav.jpg")
    self.im11 = ImageTk.PhotoImage(self.im10)
    self.img1 = Label(self, image=self.im11)
    self.img1.place(x=50, y=215)
```

④ play_audio()函数：用于播放音频。

根据音频路径加载音频文件，根据当前的播放状态执行开始播放或停止播放的操作。

```
def play_audio(self):
    mixer.music.set_volume(1)
    mixer.music.load(self.radio_path)
    if(self.play_status):
        self.bt3t = self.canvas_root.create_text((self.bt3_left+self.bt3_right)/2,(self.bt3_top+self.bt3_bottom)/2,text='播 放',font=('楷体',20,'bold'),fill='white')# 按钮显示文本
        mixer.music.stop()
    else:
        # self.bt3t = self.canvas_root.create_text((self.bt3_left+self.bt3_right)/2,(self.bt3_top+self.bt3_bottom)/2,text='停 止',font=('楷体',20,'bold'),fill='white')# 按钮显示文本
        mixer.music.play()
```

⑤ run_out()函数：执行音频的情感预测和文本识别。

调用 inference()函数进行情感预测，recognize()函数进行文本识别，将得到的结果展示在对应的文本框中。

```
def run_out(self):
    self.text1.delete(1.0, END)
    self.text2.delete(1.0, END)
    result, score = inference(self.radio_path)
    text = recognize(self.radio_path)
    self.text1.insert(1.0, result)
    self.text2.insert(1.0, text['result'][0])
```

2. 绘制特征图（normal_figure 和 mfcc_save 函数）

normal_figure()：根据音频路径读取音频，使用 librosa 库绘制音频信号的波形图，并保存在对应路径。

```
def normal_figure(wav_path):
    x, sr = librosa.load(wav_path)
    #绘制声波信号
    plt.figure(figsize=(6, 2))
    librosa.display.waveshow(x, sr=sr)
    # plt.show()
    plt.savefig("sources/wav.jpg")
```

mfcc_save()：根据音频路径读取音频，使用 librosa.feature.mfcc 函数提取 MFCC 特征，并绘制特征图保存在对应路径，图中横轴代表音频时间，纵坐标代表计算得到的 mfcc 数值，颜色代表倒频系数大小。

```
def mfcc_save(wav_path):
    # STFT
    y, sr = librosa.load(wav_path)
    S = librosa.stft(y, n_fft=2048, hop_length=None, win_length=None, window='hann', c
enter=True, pad_mode='reflect')

    # 取绝对值
    S = np.abs(S)
    print(y.shape)
    print(S.shape)

    #梅尔频率倒谱系数 MFCC
    mfccs = librosa.feature.mfcc(y=y, sr=sr)
    print(mfccs.shape)

    # MFCC:
```

```
    plt.figure(figsize=(6, 2))
    librosa.display.specshow(mfccs, sr=sr, x_axis='time')
    plt.savefig("sources/mfcc.jpg")
```

3. 语音情感预测（inference 函数）

预测准备：加载归一化文件、分类标签和训练好的情感预测模型。

```
# 加载归一化文件
scaler = joblib.load(scaler_path)
# 获取分类标签
with open(label_list_path, 'r', encoding='utf-8') as f:
    lines = f.readlines()
class_labels = [l.replace('\n', '') for l in lines]
# 获取模型
device = torch.device(device_type)
model = torch.load(model_path)
model.to(device)
model.eval()
```

根据传入的音频路径 audio_path，调用 load_audio 函数读取音频并获取音频特征，将一维数据升维并进行归一化后放入 tensor 中，利用训练好的模型执行预测，最终返回预测类别和对应得分。

```
def inference(audio_path):
    audio = load_audio(audio_path, mode='infer')
    audio = audio[np.newaxis, :]
    audio = scaler.transform(audio)
    audio = torch.tensor(audio, dtype=torch.float32, device=device)
    # 执行预测
    output = model(audio).data.cpu().numpy()[0]
    # 显示图片并输出结果最大的 label
    lab0 = np.argsort(output)[-1]
    label = class_labels[lab0]
    score = output[lab0]
    return label, score
```

4. 语音文本识别（recognize 函数）

这里使用百度语音识别 API 进行文本识别，最初想要用 Speech 实现文本识别，但尝试后发现效果不够好，发现百度 API 的识别结果准确度很高，于是最后选择了百度 API 进行实现。我们实现的这一应用目前针对的是中文的语音，这里识别时选择的语言模型为普通话，但我们训练情感预测模型时包括了等量

的中英文样本，对于中英文语音皆可实现预测情感。

这里使用百度语音识别 API 提供的客户端类 AipSpeech，创建 AipSpeech 客户端对象 client，其中参数 APP_ID、API_KEY 和 SECRET_KEY 是用于标识和验证用户身份的认证信息。

调用 get_file_content()读取音频文件，调用百度语音识别 API 进行语音识别，设置参数'wav'表示音频格式为 WAV，音频文件的采样率为 16000Hz，dev_pid 指定语言模型为普通话，返回文本识别结果。

```
APP_ID = '33091558'
API_KEY = 's3Abr6RMX3krVhWpfoRseMl1'
SECRET_KEY = 'jq6PlXFBk3FjgqpkWvBDrdAECAXFNguF'
client = AipSpeech(APP_ID, API_KEY, SECRET_KEY)

def get_file_content(filePath):
    with open(filePath, 'rb') as fp:
        return fp.read()

def recognize(audio_file):
    result = client.asr(get_file_content(audio_file), 'wav', 16000, {
        'dev_pid': 1537,
    })
    return result
```

五、 实验结果

5.1 情感分析模型训练结果

对于情感分析模型的训练，具有参考意义的一共有 4 组，其参数情况如下：

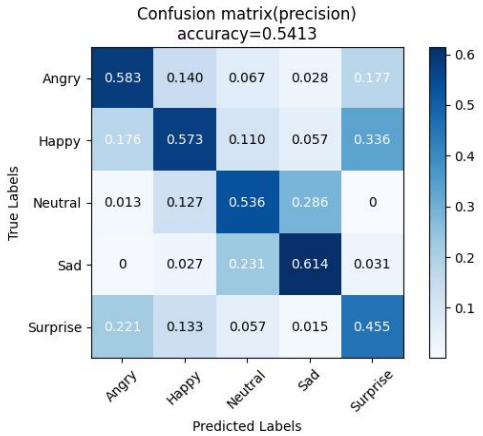
batch_size	epoch	Learning rate	评估 accuracy
512	50	1e-3	0.5413
512	100	0.01	0.8793
512	100	0.1	0.9287
512	100	0.5	0.9293

因此，最终采用的模型参数为 **batch_size=512,epoch=100,learning**

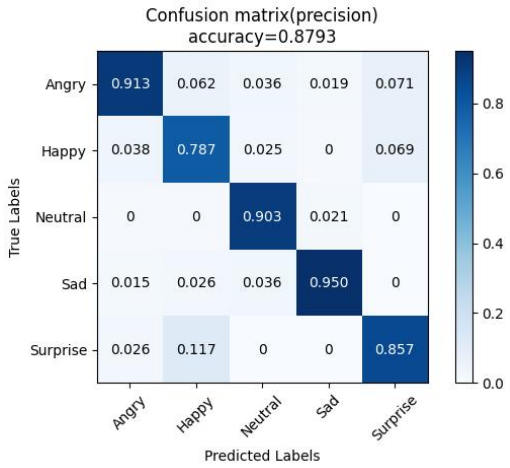
rate=0.5。

以下展示四种参数组合的混淆矩阵

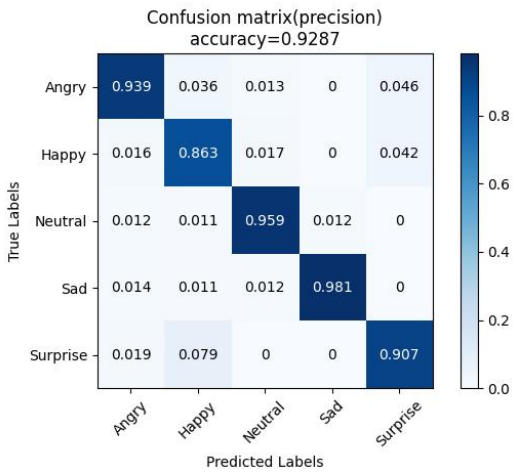
- Epoch=50，learning rate=1e-3



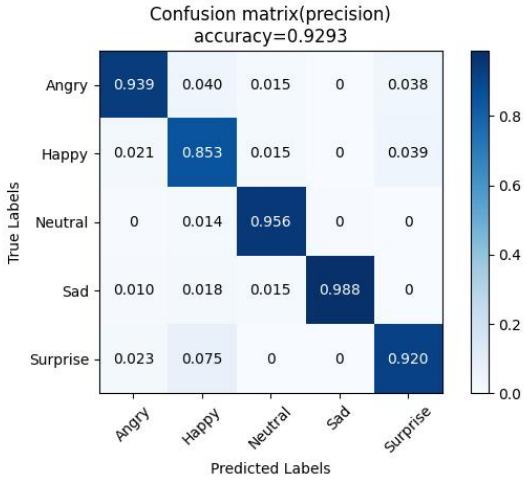
- Epoch=100，learning rate=0.01



- Epoch=100，learning rate=0.1

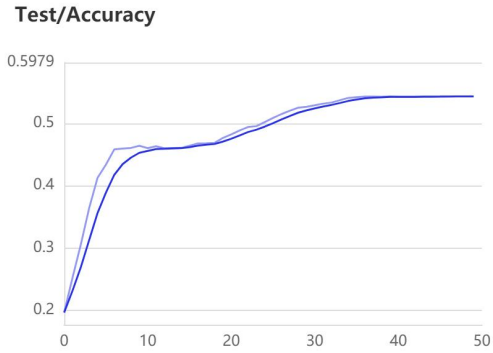


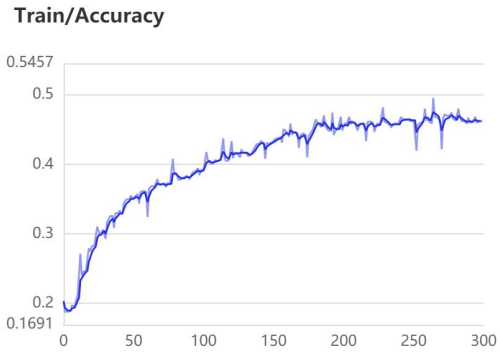
- Epoch=100，learning rate=0.5

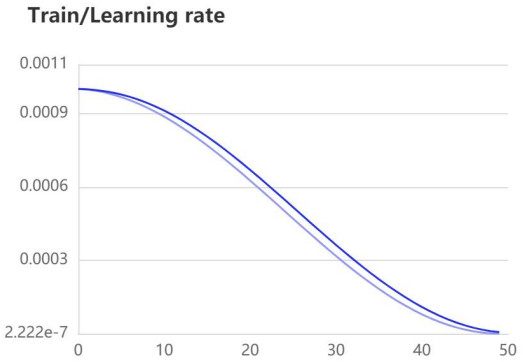


以下展示四种参数组合的可视化训练过程

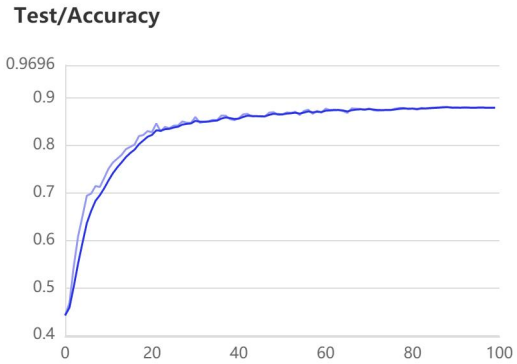
- Epoch=50，learning rate=1e-3

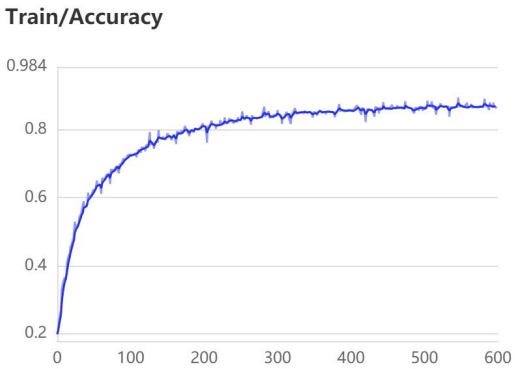


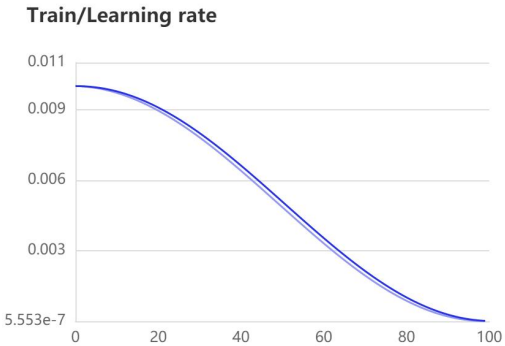


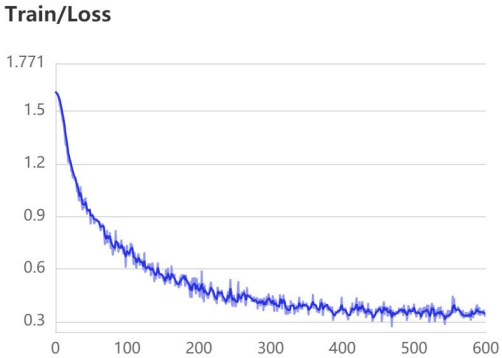


- Epoch=100，learning rate=0.01

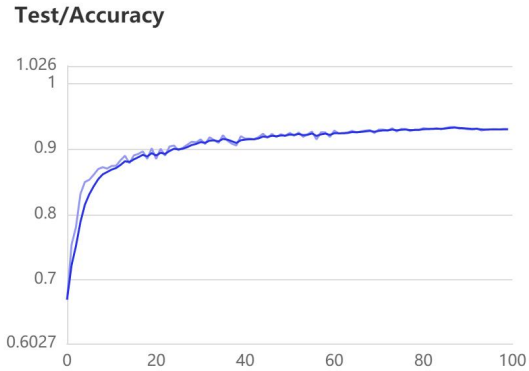


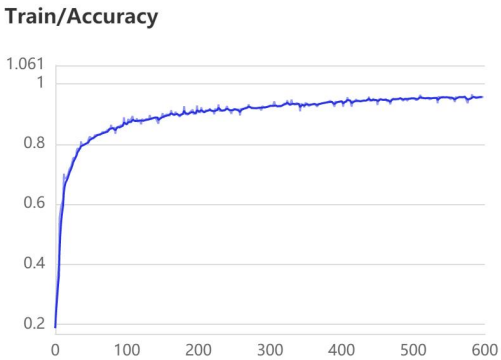


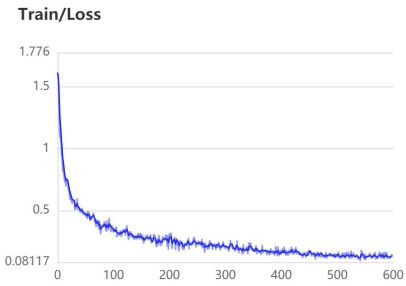


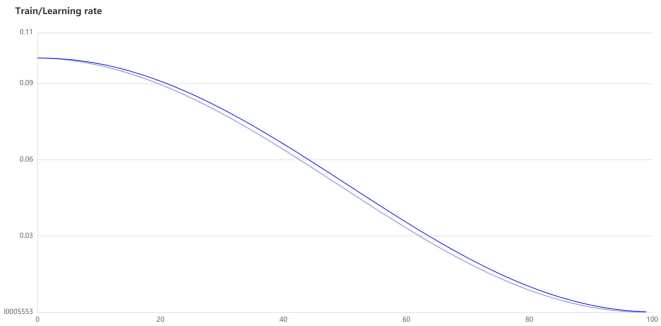


- Epoch=100，learning rate=0.1

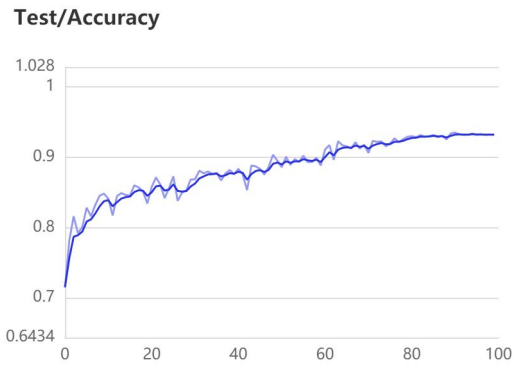


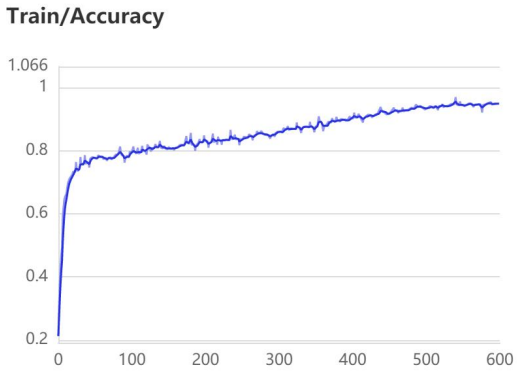


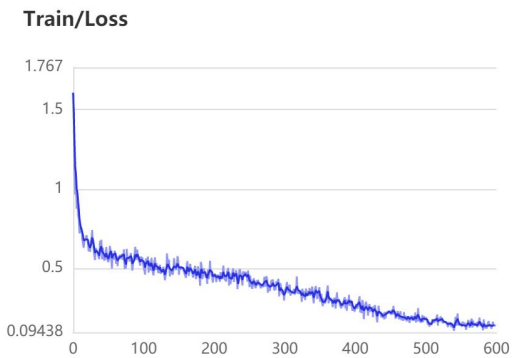


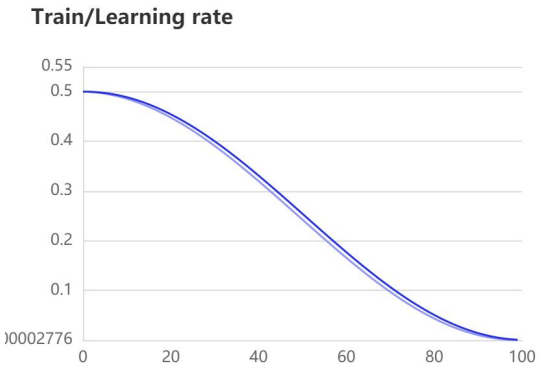


- Epoch=100，learning rate=0.5







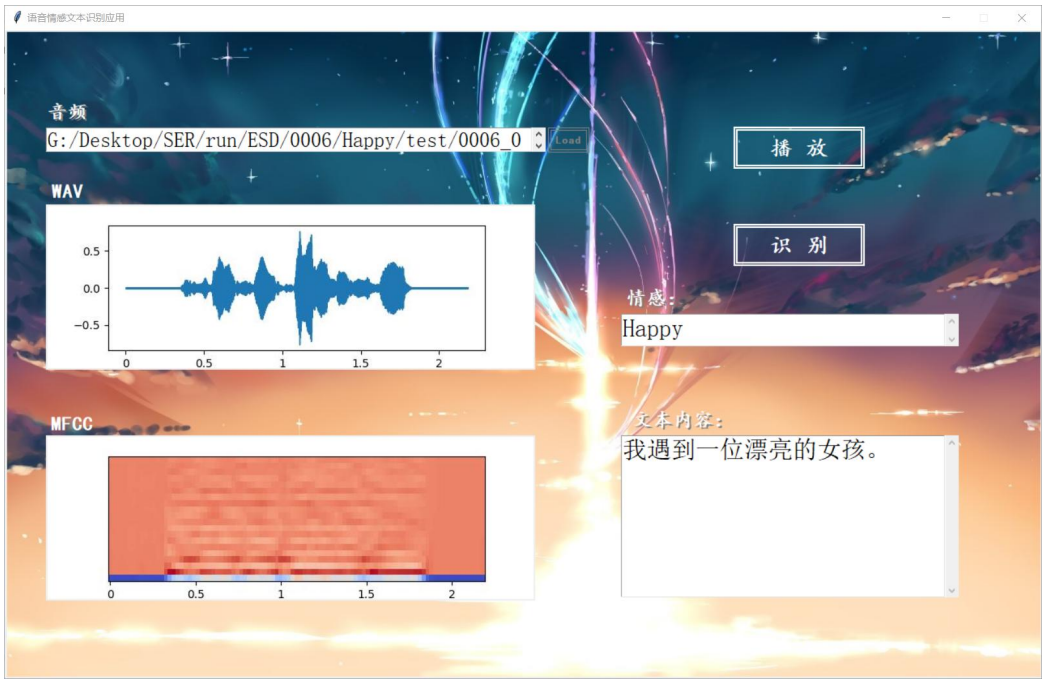


5.2 最终实现结果

GUI 的最终实现效果如下：

包括应用初始首页和功能界面两个界面，在首页点击“开始”按钮后进入功能界面。用户在功能界面选择音频文件后，将在左侧展示音频对应的波形图和 MFCC 特征图像，右侧可选择播放音频，点击识别后会在右下方文本框中得到预测的情感标签和所识别到的文本内容。

语音情感文本识别应用
开 始


语音情感文本识别应用
音频
G:/Desktop/SER/run/ESD/0006/Happy/test/0006_0
Load
播 放
WAV
0.5
0.0
−0.5
0
0.5
1
1.5
2
识 别
情感：
Happy
MFCC
文本内容：
我遇到一位漂亮的女孩。

六、 实验总结

6.1 小组分工

- **樊星:统筹规划、开题与结题答辩、模型定义、导出预测模型代码编写、GUI 代码编写、实验报告撰写**
- 唐 XX:调参测试、开题与结题资料搜集、数据读取与特征提取部分代码编写、实验报告撰写
- 郭 XX:开题与结题资料搜集、PPT 制作、数据增强器、模型训练部分代码编写、实验报告撰写
- 丁 XX:调参测试、开题与结题资料搜集、模型评估、评估工具类、数据工具类代码编写、实验报告撰写

6.2 实验心得

本次实验我主要负责模型的定义和 GUI 部分的实现。其中，模型的结构主要参考了论文《一种基于双向 LSTM 的语音情感识别模型》，这一部分很顺利，但是随后预测模型的导出和 GUI 的实现部分就有一堆问题了。问题主要来自于 tkinter 丑陋的原始组件和及其难用的组件排版方法。Tkinter 最常用的组件排版方法为 grid 布局，这是一种网格化的布局方式，在创建对话框等小巧的界面时很好用，但是我们要在界面上铺图片和文本框，用 grid 布局的话就会导致各种组件的畸形。所以就只能使用 place 布局，在二维坐标系中防止组件。但这个 place 布局的所有坐标参数都需要自己设定，这在非编辑式的 UI 设计中是非常麻烦的，在调整位置时只能一点一点去调坐标参数来将组件放置在合适的位置，这就是 place 布局很少使用的原因。而且这种布局方式最反人类的一点就是它画布上的文字的坐标不是文字左上角的起点坐标，而是文字中心的坐标，这就导致每种不同大小、长度的文字都需要单独再调整坐标来让它放到合适的位置。除此之外，tkinter 的按键无法设置为透明，这就导致按键在我们的 UI 上非常突兀，于是我们就通过在画布上画矩形框和文字并绑定相应区域的相应函数的方式实现了透明按键。总而言之，本次实验让我更加熟悉了 BiLSTM 模型，并且深切体会到了调整前端 UI 的麻烦之处，看来还是后端开发更适合我。

七、 参考资料

1. Zhou K, Sisman B, Liu R, et al. Emotional voice conversion: Theory, databases and esd[J]. Speech Communication, 2022, 137: 1-18.
2. TAO Jianhua, CHEN Junjie, LI Yongwei. Review on Speech Emotion Recognition[J]. JOURNAL OF SIGNAL PROCESSING, 2023, 39(4): 571-587. doi: 10.16798/j.issn.1003-0530.2023.04.001 语言情感识别综述
3. 程适,骆晓宁,李冬城等.一种基于双向 LSTM 的语音情感识别模型[J].长江信息通信,2022,35(07):19-22.